Imagine you are a software engineer and you are supposed to create an app in python. This app will connect student to each other. The goal with this app is to make sure students at evening/nights can walk safely in campus so this app will bring walking partner to students who might need it.

In the app students can request walking partner and their location will be shared with the walking partner. After a match, the requester and the walking partner can chat with each other in the app. Also there can be an issue that maybe at some point too many requests can be submitted. The app must only send the neccessary information that the walking partner needs to know so not more and not less than what they need. The system must still working when there are many students using it and there is pressure on the servers.

Show less

Thought for 15s

I would treat this as a **student safety matching app** with three main goals:

1. match students who need a walking partner
2. protect student privacy
3. keep working even when many students use it at the same time

Here is how I would design it in Python.

1. Main functions of the app

Student can:

- sign in with university account
- request a walking partner
- share current location for the walk
- see when a partner is found
- chat with the matched partner
- end the walk

- report problems or unsafe behavior

Walking partner can:

- receive nearby requests
- accept or reject a request
- see only the needed information
- chat with the requester
- follow the route during the walk
- mark the walk as completed

Admin/security team can:

- monitor active requests
- see system health
- handle reports
- manage misuse or fake accounts

2. Functional requirements

These are the things the system **must do**.

Matching

- The system shall allow a student to create a walk request.
- The system shall find available walking partners near the requester.
- The system shall send the request to suitable partners.
- The system shall confirm the match when one partner accepts.

Location sharing

- The system shall share the requester's live location only with the matched walking partner.
- The system shall stop location sharing when the walk ends or is cancelled.
- The system shall update location in near real time.

Chat

- The system shall allow the requester and walking partner to chat after a match.
- The system shall store chat only as long as needed by policy.

Safety

- The system shall allow users to cancel a request.
- The system shall allow users to report danger, abuse, or suspicious behavior.
- The system shall support an emergency button if needed.

3. Important quality requirements

Your text also describes strong **non-functional requirements**.

Privacy / data minimization

This is very important.

The app should send **only the necessary information**.

For example, the walking partner may need:

- first name
- profile photo maybe
- pickup location
- destination area
- live location during the walk
- university verification status

They probably do **not** need:

- personal phone number
- full home address
- student ID number
- full schedule
- exact personal details not related to the walk

So the system should use **least data possible**.

Scalability

The system must still work when many students use it.

This means:

- many requests at the same time
- many location updates
- many chat messages
- no crashing when load increases

Performance

Students need quick matching, so:

- request submission should be fast
- matching should happen quickly
- chat should feel instant

- location updates should not lag too much

Availability

The system should still be usable at night when students need it most.

Security

The app handles location and safety data, so:

- strong login
- encrypted communication
- secure storage
- access control
- audit logs

4. Good Python-based system design

I would split it into small services or modules.

Backend in Python

A good choice would be:

- **FastAPI** for REST API
- **WebSockets** for chat and live location
- **PostgreSQL** for main data
- **Redis** for fast matching, caching, and queue handling
- **Celery / background workers** for heavy tasks
- **Docker** for deployment

Main modules

- **Auth service**: login with university account
- **Request service**: create/cancel walk requests
- **Matching service**: find the best walking partner
- **Location service**: handle live location updates
- **Chat service**: real-time messages
- **Notification service**: push notifications
- **Safety/reporting service**: reports and emergency events

5. How matching can work

A simple matching logic:

1. student sends a request
2. system checks:
   - current location
   - destination
   - time
   - available partners nearby
3. system ranks possible partners
4. system offers request to one or more suitable partners
5. first accepted match becomes active
6. chat and location sharing start

To avoid bad matches, the system can also check:

- max walking distance for partner
- same campus zone
- current availability
- trust level / verified account

6. How to handle too many requests

This is one of the most important architecture problems.

If too many students send requests, I would do this:

Use queues

Instead of making every request processed immediately in one place, put requests in a queue.

That helps because:

- the server does not get overloaded at once
- requests are handled in order
- workers can process requests in parallel

Scale horizontally

Run multiple backend instances behind a load balancer.

That means:

- more servers can share the work
- one server failure does not break the whole app

Use Redis

Redis is useful for:

- temporary active requests
- fast matching lookup
- rate limiting
- session/state caching

Limit unnecessary updates

Location updates can be expensive.

So:

- send updates every few seconds, not every millisecond
- only send when user moved enough distance
- stop updates after walk ends

Rate limiting

Prevent spam and abuse:

- limit how many requests one user can create in a short time
- limit chat flood
- limit too many reconnects

Graceful degradation

If demand is too high:

- show estimated waiting time
- put request in waiting queue
- send “no partner available right now”
- maybe escalate to campus security if supported

7. Privacy by design

Since the app uses location, privacy must be built in from the beginning.

Data minimization

Only collect:

- name
- university verification
- live location during active request
- destination/pickup info
- chat during active case

- safety logs if needed

Access control

Only matched users can see each other's relevant info.

Time limits

- location data should expire quickly
- chat should not be kept forever unless policy needs it
- inactive requests should auto-delete

Encryption

- HTTPS/TLS for all traffic
- encrypted sensitive data in storage where needed

Consent

User should know:

- what is shared
- when it is shared
- with whom it is shared
- when it stops

8. Example architecture drivers

If this is for a software engineering or architecture task, the strongest drivers are:

Functional drivers

- request a walking partner
- match students
- share live location
- chat after match

Quality drivers

- privacy
- security
- scalability
- performance
- availability

These drivers will strongly affect the architecture.

For example:

- **privacy** pushes us to strict access control and minimum data sharing
- **scalability** pushes us to queues, Redis, and multiple backend instances
- **real-time chat/location** pushes us to WebSockets
- **security** pushes us to strong authentication and encrypted transport

9. Example quality attribute scenarios

These are useful if you need software architecture answers.

Scalability scenario

When 5,000 students use the app during late evening, the system should still accept new requests and complete matching within an acceptable time, for example under 5 seconds for request registration and under 30 seconds for first partner offer.

Privacy scenario

When a student is matched, the system should reveal only the minimum data needed for the walking partner to complete the walk safely, and no other student should be able to access that location or chat.

Availability scenario

If one backend server fails during peak hours, the system should continue serving students using other running instances without complete service interruption.

Performance scenario

When a matched student sends a chat message, the other student should receive it almost immediately, for example within 1 second under normal load.

Security scenario

If an unauthorized user tries to access another student's live location, the system should deny the request and log the attempt.

10. Short summary of the best solution

A good Python solution would be:

- **FastAPI backend**
- **PostgreSQL** for persistent data
- **Redis** for fast temporary data and queues
- **WebSockets** for chat and live location
- **background workers** for matching and notifications

- **load balancer + multiple app instances** for high traffic
- **strict privacy rules** so only necessary data is shared

So the app is not only functional, but also:

- safe
- private
- scalable
- fast

If you want, I can also turn this into a **system architecture answer for an exam/assignment**, with clear sections like **functional requirements, non-functional requirements, architecture drivers, and design decisions**.

Now write the app in paython and give me a working code in .py

I'm turning this into a runnable Python MVP in one `.py` file: a FastAPI backend with matching, live location updates, and chat. After that I'll check that it parses cleanly and give you the file plus run steps.

The backend file is ready and it passes a Python syntax check. I'm doing one more smoke test now so I can confirm the main flow works: register, login, match, chat, and end walk.

Thought for 5m 3s

Done — here is the Python file:

Download `campus_walk_safe_app.py`

It is a working **FastAPI MVP backend** in one `.py` file. It includes:

- register and login
- mark a student as available to walk
- create a walking request
- auto-match with the nearest available walking partner
- live location updates
- in-app chat after match
- end walk
- basic rate limiting for too many requests
- privacy-minded responses with only necessary shared data

I also smoke-tested the main flow successfully:

- register
- login
- set availability
- create request
- match
- chat
- end walk

Run it like this:

Bash

```
pip install fastapi uvicorn
uvicorn campus_walk_safe_app:app --reload
```

Then open:

Bash

```
http://127.0.0.1:8000/docs
```

Important:

- this is a **demo/MVP backend**
- data is stored **in memory**, so it resets when the server restarts
- it is not yet production-ready with database, real auth, push notifications, or mobile UI

The next useful step would be to build a simple frontend for it in Python or React.

Now refactore or expand the code based on these requirements:

1. Error handling: The program shall be safe in any possible event.
2. The students shall be able to login into their account and manage to see their history of requests.
3. The system must be safe and encrypt the data.
4. The chat system must be saved and the user can chat to each other from the point they leave the chat.
5. The gps must be used only when the request is sent and

Show more

I'm updating the backend instead of patching it blindly. First I'm reading the current file and then I'll rebuild the parts that affect security, request history, persistent chat, GPS rules, and emergency sharing.

I found one design conflict in the requirements: earlier version used live location during the walk, but now you want GPS only at request time and on arrival. I'm refactoring the flow so location becomes checkpoint-based: start location, optional safety sharing, and arrival confirmation instead of continuous tracking.

The new version compiles. I'm doing a real API smoke test now with a temporary database so I can confirm the upgraded features actually work together and not only on syntax.

Thought for 11m 32s

Updated version is ready:

Download `campus_walk_safe_app_v2.py`

What changed:

- stronger error handling with global exception handlers
- persistent SQLite storage instead of only memory
- login + request history
- encrypted sensitive data at rest
- salted password hashing
- saved chat history that users can reopen later
- GPS only at:
  - request creation
  - arrival confirmation
- safety contacts for family/friends
- public safety feed token for trusted contacts to see shared checkpoints

How to run:

Bash

```
pip install fastapi uvicorn cryptography email-validator
uvicorn campus_walk_safe_app_v2:app --reload
```

Then open:

Bash

```
http://127.0.0.1:8000/docs
```

I also smoke-tested this flow successfully:

- register
- login
- add safety contact
- set walker availability
- create request
- auto-match
- send/read chat
- confirm arrival
- read request history
- read public safety feed for a trusted contact

One important note:
No software can truly be safe in **every possible real-world event**, but this version is much safer on the software side and handles failures much better.

If you want, I can next make:

- a simple frontend for this backend, or
- a cleaner project structure with multiple `.py` files instead of one large file.